

Trabajo Práctico 2: Tecnología Digital VI

Predicción de presionar “Forward” en Spotify

Facundo Amaya

Juan Ignacio Elosegui

Nicolás Boudourian

Universidad Torcuato Di Tella
Segundo cuatrimestre 2025

Resumen

En este trabajo se desarrolla un sistema de aprendizaje automático para predecir si un usuario presionará el botón *forward* en una canción de Spotify, a partir de información de sesiones de escucha. Se presentan las etapas de análisis exploratorio, creación de variables, estrategia de validación, modelado y análisis de importancia de atributos.

1. Introducción

En este trabajo se desarrolló un sistema de aprendizaje automático para predecir si un usuario de Spotify presionará el botón *forward* durante la reproducción de una canción. El problema se aborda como una tarea de clasificación binaria, donde la variable objetivo toma el valor 1 cuando `reason_end = "fwdbtn"` y 0 en caso contrario.

La métrica de evaluación utilizada es el **ROC-AUC**, dado que mide la capacidad discriminativa del modelo independientemente del umbral de decisión. El enfoque adoptado combina ingeniería de variables temporales y de comportamiento del usuario, validación temporal y un modelo basado en **XGBoost**, un algoritmo de gradient boosting sobre árboles de decisión.

2. Análisis Exploratorio de Datos (EDA)

El dataset original provisto por la competencia se encuentra dividido en dos archivos:

- `train_data.txt`: contiene observaciones históricas con la variable `reason_end`.
- `test_data.txt`: idéntico formato pero sin la columna `reason_end`.

Cada fila representa una reproducción de una canción por un usuario identificado por `username`, con información asociada como hora, país de conexión, plataforma y metadatos de la canción.

Durante el análisis exploratorio se detectaron:

- Variables categóricas de alta cardinalidad (`master_metadata_track_name`, `master_metadata_album` etc.).
- Campos booleanos (`shuffle`, `offline`, `incognito_mode`).
- Campos temporales (`ts`, `offline_timestamp`) a partir de los cuales se construyeron patrones temporales.

Patrones relevantes observados:

- Mayor frecuencia de *forward* en los primeros temas de una sesión.
- Comportamiento diferenciado entre fines de semana y días hábiles.
- Usuarios con historial previo de saltos tienden a repetir ese comportamiento.

Además, encontramos otros dos patrones:

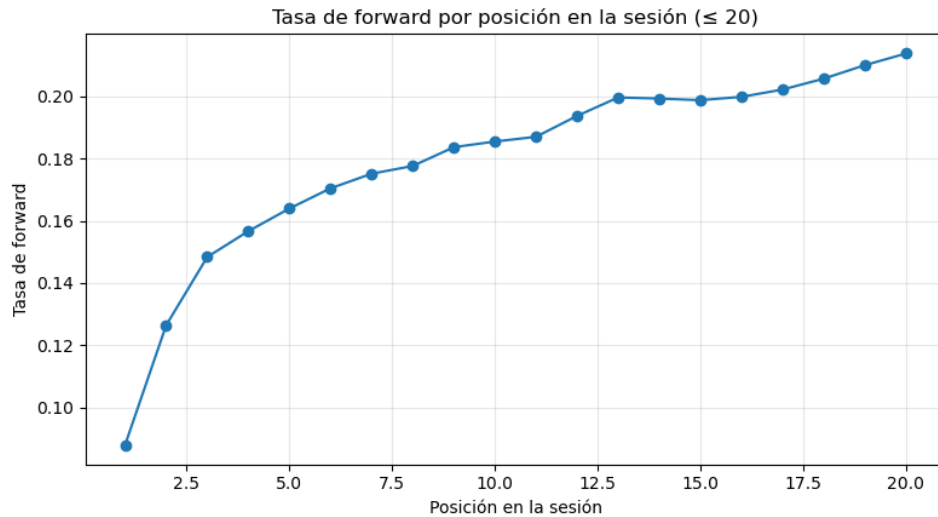


Figura 1: Tasa de forward por posición en la sesión.

La tasa de forward (probabilidad de que el usuario salte la canción) aumenta a medida que avanza la sesión. Al principio (posición 1–3) los usuarios escuchan más tiempo y saltan menos, pero después de varios temas la probabilidad de salto se incrementa progresivamente, alcanzando un 21 % en la posición 20.

Al comienzo de una sesión el usuario suele estar más activo eligiendo música que le gusta o ajustando la lista; más adelante, puede distraerse, cansarse o buscar variedad, lo que genera más saltos. También puede reflejar que las canciones al inicio son de mayor preferencia o que el modelo de recomendación las ordena según relevancia.



Figura 2: Tasa de forward por hora del día.

Se observan fluctuaciones horarias: menor tasa de saltos entre las 3 y las 6 h, picos cerca de las 10 h y entre las 19–22 h. Las horas nocturnas y de noche presentan más saltos, mientras que la madrugada y el mediodía son más estables o con tasas más bajas. Durante la mañana o el trabajo, los usuarios suelen dejar la música de fondo y saltan menos. En cambio, al final del día (tarde/noche), escuchan más activamente y seleccionan

canciones, generando más forwards. Los picos de la noche pueden coincidir con ocio o actividad social.

3. Creación de Variables

Se implementó un proceso de **feature engineering** que combina información temporal, de usuario y de artista. Las principales variables creadas fueron:

Cuadro 1: Principales variables derivadas en el pipeline.

Variable	Fuente	Descripción
hour, dow	ts	Hora y día de la semana de reproducción
hour_sin, hour_cos, dow_sin, dow_cos	ts	Representación cíclica de hora y día
secs_since_prev	Diferencia entre reproducciones	Segundos desde la última reproducción del mismo usuario
log1p_secs_since_prev	secs_since_prev	Escala logarítmica para valores extremos
user_forward_rate_hist	Historial usuario	Promedio histórico de forwards (suavizado bayesiano)
user_roll_mean{3, 5, 10}	Historial reciente	Promedio móvil de los últimos 3, 5 y 10 eventos
secs_since_last_forward	Eventos previos	Tiempo desde el último <i>forward</i>
user_artist_rate_hist	user-artist	Probabilidad histórica de forward para un artista específico
artist_freq	Entrenamiento	Frecuencia global del artista (popularidad)
session_id, pos_in_session	ts	Segmentación en sesiones (gaps > 30 min) y posición dentro de la sesión
is_first_in_session	pos_in_session	Indicador de primer tema en la sesión

También se aplicaron transformaciones trigonométricas para capturar periodicidad diaria y semanal, y codificación categórica mediante **OneHotEncoder** dentro de un **ColumnTransformer**.

4. Construcción del Conjunto de Validación

Dado que el comportamiento de los usuarios y la secuencia temporal son relevantes, se optó por un **split temporal** para validación. Se utilizó el 90 % inicial de las observaciones (ordenadas por **ts**) para entrenamiento y el 10 % más reciente para validación:

$$\text{train: } t < q_{0,9}(\text{ts}) \quad ; \quad \text{valid: } t \geq q_{0,9}(\text{ts})$$

Este esquema evita fuga temporal (*data leakage*) entre entrenamiento y validación, y emula la dinámica real de predicción sobre sesiones futuras.

5. Modelos y Búsqueda de Hiperparámetros

El modelo final utilizado fue un **XGBoost Classifier** con las siguientes configuraciones principales:

Parámetro	Valor
n_estimators	3000
learning_rate	0.03
max_depth	4
min_child_weight	4
subsample, colsample_bytree	0.9
reg_lambda, reg_alpha	1.0, 0.1
early_stopping_rounds	100
tree_method	hist

Cuadro 2: Configuración de hiperparámetros del modelo XGBoost.

Se evaluaron distintos modelos de clasificación (árboles de decisión, regresión logística y métodos de boosting). Seleccionamos **XGBoost** finalmente por su excelente desempeño en tareas con datos tabulares heterogéneos, por su capacidad para manejar relaciones no lineales y variables con distintas escalas, y por contar con regularización interna que reduce el riesgo de sobreajuste. Además, ofrece alta eficiencia computacional gracias a su implementación paralelizada y soporte para *early stopping*, lo que permite ajustar los hiperparámetros de manera precisa y rápida.

Se aplicó **early stopping** monitoreando el AUC en el conjunto de validación temporal. El modelo se entrenó con una ponderación inversa de clases (**scale_pos_weight**) para compensar el desbalance entre eventos de forward y no-forward.

El resultado de validación obtenido fue aproximadamente:

$$\text{AUC}_{\text{valid}} \approx 0,78$$

El conjunto de test se transformó con el mismo preprocesador y se generó un archivo `modelo_xgb.csv` con las probabilidades predichas de *forward*.

6. Importancia de Atributos

El análisis de importancia de variables (*feature importance*) reveló que los predictores más influyentes son los relacionados al historial del usuario y su comportamiento temporal:

- **user_forward_rate_hist**: predictor dominante, refleja la propensión del usuario a saltar canciones.
- **secs_since_prev**: tiempos largos entre reproducciones reducen la probabilidad de forward.
- **pos_in_session**: los primeros temas presentan mayor probabilidad de salto.
- **user_artist_rate_hist**: mide la afinidad usuario-artista.

- `hour_sin` / `hour_cos`: reflejan patrones horarios de escucha.

En general, las variables temporales y de historial del usuario aportaron más información que las categóricas (`platform`, `conn_country`, etc.).

7. Conclusiones

El sistema propuesto alcanza un desempeño sólido ($AUC \approx 0,78$) utilizando un pipeline reproducible y eficiente basado en XGBoost. Las claves del buen rendimiento fueron:

- Un **feature engineering** centrado en comportamiento temporal y usuario–artista.
- Un **esquema de validación temporal** que evita fugas de información.
- Un **modelo robusto y rápido**, con *early stopping* y regularización moderada.

Limitaciones:

- No se incorporaron embeddings ni variables de audio de la API de Spotify.

Futuras mejoras:

- Integrar información externa (popularidad, duración, audio features).

Referencias

- Kaggle Competition: <https://www.kaggle.com/t/1ec753ebddb246ee86e6731ec097c6e1>
- Spotify API Documentation: <https://developer.spotify.com/documentation/web-api>